TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 10: Add Your Own System Call: trace

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Part 1: The number and the user side **5**

1.1 User-space declarations, stub, and utility 5

1.2 Check 1: The number reaches the kernel 5

2 Part 2: The kernel side **7**

2.1 Per-process mask, handler, dispatch table, and `fork` inheritance 7

2.2 Check 2: `trace` works 7

3 Part 3: Play with the mask **9**

3.1 3.1 All the calls (all bits on) 9

3.2 3.2 Only `fork` (bit 1, mask 2) on `usertests forkforkfork` 9

3.3 3.3 Only `write` (bit 16, mask 65536) 10

3.4 Answers to Part 3 observations 11

Conclusions **12**

ABSTRACT

The goal of this laboratory is to design and implement a new system call, `trace` (system call number 23), in the xv6 RISC-V operating system kernel, together with a user-space utility `trace` that enables selective system-call tracing via a bit mask. In Part 1, the system call number, user declaration, assembly stub generator (`usys.pl`), and user program (`user/trace.c`) are added. In Part 2, per-process mask storage (`trace_mask`), the `sys_trace` kernel handler, dispatch and name tables in `kernel/syscall.c`, and mask inheritance across `fork()` in `kernel/proc.c` are implemented. In Part 3, the implementation is verified across multiple bit masks, including all calls (`2147483647`), fork-only stress testing (`trace 2 usertests forkforkfork`), and write-only tracing (`trace 65536 echo hi`). **Keywords:** xv6, RISC-V, system call, SYS_trace, syscall dispatch, fork inheritance, bit mask, usertests

INTRODUCTION

Goal of the laboratory (Scopul lucrării)

The purpose of this laboratory work is to modify the xv6 RISC-V operating system kernel by adding a new system call, `trace`, and its companion user-space command `trace mask command [args...]`. Whereas in Lab 9 `strace` observed system calls from outside on Linux, in this laboratory we build the tracing mechanism inside the kernel itself, following system call number 23 from a user program down to the kernel trap handler. Specifically, the objectives are:

1. **Part 1 (The number and the user side):** Assign syscall number 23 (`SYS_trace`) in `kernel/syscall.h`, declare `int trace(int);` in `user/user.h`, add the `ecall` stub entry in `user/usys.pl`, implement `user/trace.c`, add `$U/_trace` to `Makefile`, and verify that syscall 23 reaches the kernel.
2. **Part 2 (The kernel side):** Add `int trace_mask` to `struct proc` in `kernel/proc.h`, implement `sys_trace(void)` in `kernel/sysproc.c`, register `sys_trace` and `syscallnames[]` in `kernel/syscall.c` to print traced calls with `printk`, propagate `trace_mask` from parent to child in `fork()` (`kernel/proc.c`), and verify `trace 32 grep hello README`.
3. **Part 3 (Play with the mask):** Test and analyze different masks: tracing all system calls (`trace 2147483647 grep hello README`), tracing `fork` across child processes during process-table exhaustion (`trace 2 usertests forkforkfork`), and tracing `write` calls (`trace 65536 echo hi`).

Because the course's newer `xv6-riscv` tree already assigns syscall 22 to `SYS_sync` and names the kernel console printing function `printk`, our implementation uses `SYS_trace = 23` and `printk`. The complete git diff is provided in `xv6-trace.patch`.

1 PART 1: THE NUMBER AND THE USER SIDE

1.1 User-space declarations, stub, and utility

Five edits wire the `trace` system call from user space to the `ecall` instruction:

1. **`kernel/syscall.h`:** Define the system call number at the end of the table:

```
#define SYS_sync   22
#define SYS_trace  23
```

2. **`user/user.h`:** Add the user-space prototype next to the other system call declarations:

```
int sync(void);
int trace(int);
```

3. **`user/usys.pl`:** Register the stub entry so the Perl generator emits `li a7, SYS_trace; ecall; ret` in `user/usys.S`:

```
entry("sync");
entry("trace");
```

4. **`user/trace.c`:** Create the user program that enables tracing on the current process with `trace(atoi(argv[1]))` and then replaces its program image with the target command via `exec(argv[2], argv + 2)` (preserving the process and its `trace_mask`):

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

5. **`Makefile`:** Add `$U/_trace\` to `UPROGS` so `trace` is compiled and placed into the xv6 filesystem image.

1.2 Check 1: The number reaches the kernel

Booting xv6 with `make qemu` and running `trace 32 grep hello README` before adding the kernel handler produces:

```
$ trace
usage: trace mask command [args...]
$ trace 32 grep hello README
4 trace: unknown sys call 23
trace: failed
```

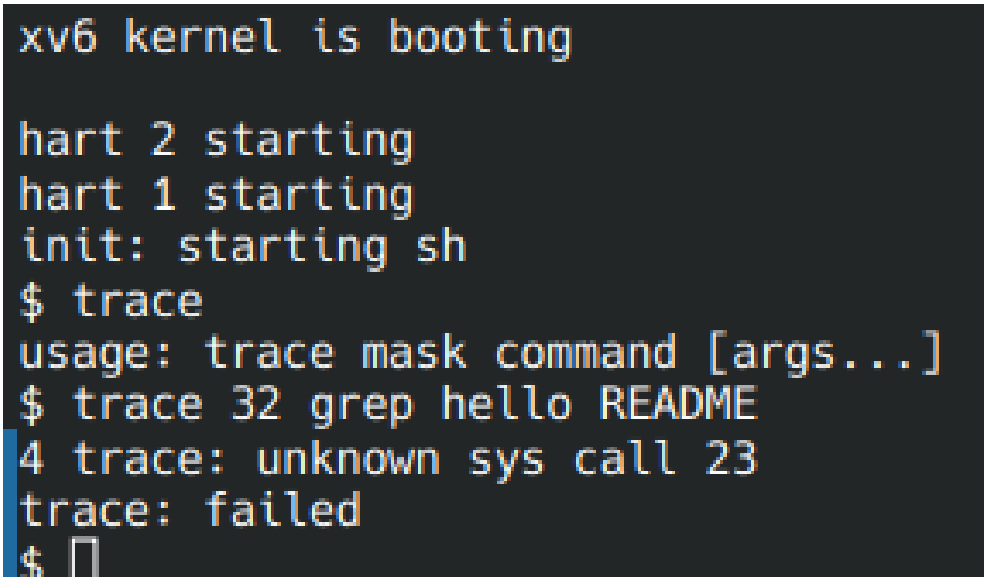


Figure 1.1 – Check 1: `trace` usage check and `unknown sys call 23` before adding the kernel handler

This confirms that the user stub loaded `a7 = 23` and trapped into the kernel via `ecall`.

2 PART 2: THE KERNEL SIDE

2.1 Per-process mask, handler, dispatch table, and `fork` inheritance

Five kernel changes complete the `trace` system call:

1. **`kernel/proc.h`:** Add `int trace_mask;` to `struct proc` so each process stores its own trace bit mask:

```
  char name[16];               // Process name (debugging)
  int trace_mask;              // Which calls to print
```

2. **`kernel/sysproc.c`:** Implement `sys_trace(void)`, which retrieves argument 0 from register `a0` via `argint(0, &mask)` and stores it in `myproc()->trace_mask`:

```
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

3. **`kernel/syscall.c` (dispatch and names):** Declare `extern uint64 sys_trace(void);`, map `[SYS_trace] = sys_trace` in `syscalls[]`, add the `syscallnames[]` lookup table above `syscall()`, and print the trace line after `syscalls[num]()` returns:

```
static char *syscallnames[] = {
  [SYS_fork] "fork",   [SYS_exit] "exit",     [SYS_wait] "wait",
  [SYS_pipe] "pipe",   [SYS_read] "read",     [SYS_kill] "kill",
  [SYS_exec] "exec",   [SYS_fstat] "fstat",   [SYS_chdir] "chdir",
  [SYS_dup] "dup",     [SYS_getpid] "getpid", [SYS_sbrk] "sbrk",
  [SYS_pause] "pause", [SYS_uptime] "uptime", [SYS_open] "open",
  [SYS_write] "write", [SYS_mknod] "mknod",   [SYS_unlink] "unlink",
  [SYS_link] "link",   [SYS_mkdir] "mkdir",   [SYS_close] "close",
  [SYS_sync] "sync",   [SYS_trace] "trace",
};
```

   And inside `syscall()`:

```
    p->trapframe->a0 = syscalls[num]();
    if ((p->trace_mask >> num) & 1)
      printk("%d: syscall %s -> %ld\n",
             p->pid, syscallnames[num], p->trapframe->a0);
```

4. **`kernel/proc.c`:** Copy the parent's trace mask to the child inside `fork()` (`kfork()`):

```
  safestrcpy(np->name, p->name, sizeof(p->name));
  np->trace_mask = p->trace_mask;
```

2.2 Check 2: `trace` works

Rebuilding and running `trace 32 grep hello README` in QEMU produces:

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

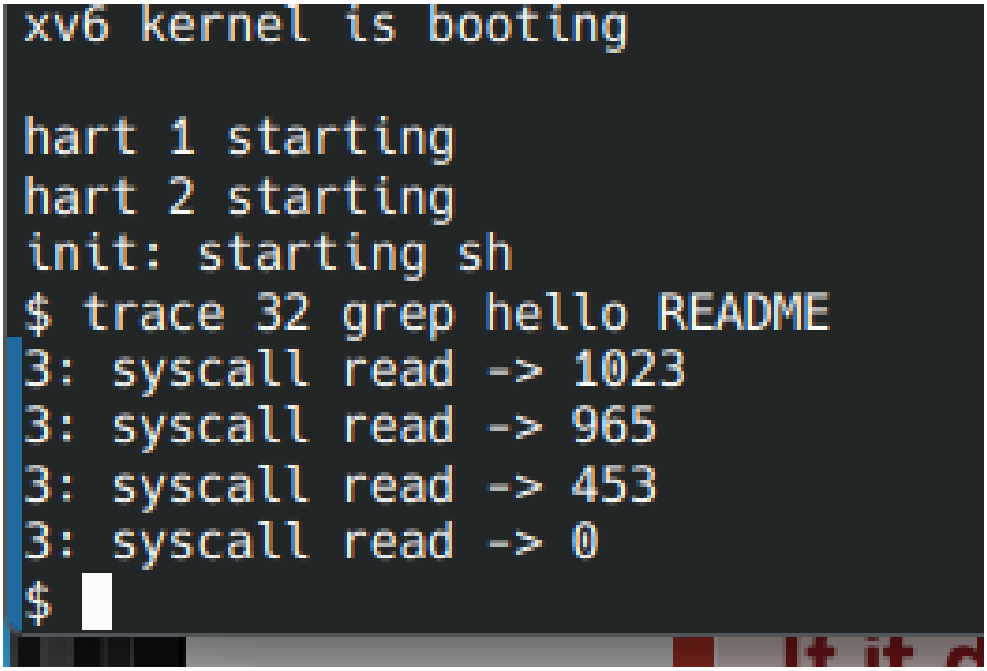


Figure 2.1 – Check 2: `trace 32 grep hello README` logging the four `read` calls

The three positive read counts sum to $1023 + 965 + 453 = 2441$ bytes (the exact size of `README`), followed by `0` for end-of-file.

3 PART 3: PLAY WITH THE MASK

3.1 3.1 All the calls (all bits on)

Passing mask `2147483647` ($2^{31} - 1$, all 31 positive bits set) traces every system call made by the process from the moment `trace()` sets the mask:

```
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open -> 3
4: syscall read -> 1023
4: syscall read -> 965
4: syscall read -> 453
4: syscall read -> 0
4: syscall close -> 0
```

Here `trace` itself returns `0`, `exec` replaces the process image with `grep` (returning `argc = 3` in `a0`), `open("README", 0)` returns descriptor `3`, four `read` calls consume the file until EOF (`0`), and `close(3)` returns `0` before `exit` (which does not return).

3.2 3.2 Only `fork` (bit 1, mask 2) on `usertests forkforkfork`

Since `SYS_fork = 1`, mask $2^1 = 2$ traces only `fork`. Running `trace 2 usertests forkforkfork` executes the `forkforkfork` stress test, which repeatedly forks processes until the xv6 process table (`NPROC = 64`) is full:

```
$ trace 2 usertests forkforkfork
usertests starting
test forkforkfork: 6: syscall fork -> 7
7: syscall fork -> 8
8: syscall fork -> 9
8: syscall fork -> 10
...
8: syscall fork -> 67
8: syscall fork -> -1
OK
ALL TESTS PASSED
```

In this run:

- Process **6** (`usertests`) forks child PID **7** to isolate the `forkforkfork` test (`6: syscall fork -> 7`);
- Process **7** forks child PID **8** (`7: syscall fork -> 8`);
- Process **8** and its children continue calling `fork()` in a loop until all free slots in `proc[]` are exhausted (reaching PID **67**), after which subsequent `fork()` calls fail and return `-1` (`8: syscall fork -> -1`). Once the children exit and are reaped, the test prints `OK` and `ALL TESTS PASSED`.

3.3 3.3 Only `write` (bit 16, mask 65536)

Since `SYS_write = 16`, mask $2^{16} = 65536$ traces only `write` system calls:

```
$ trace 65536 echo hi
hi69: syscall write -> 2

69: syscall write -> 1
```

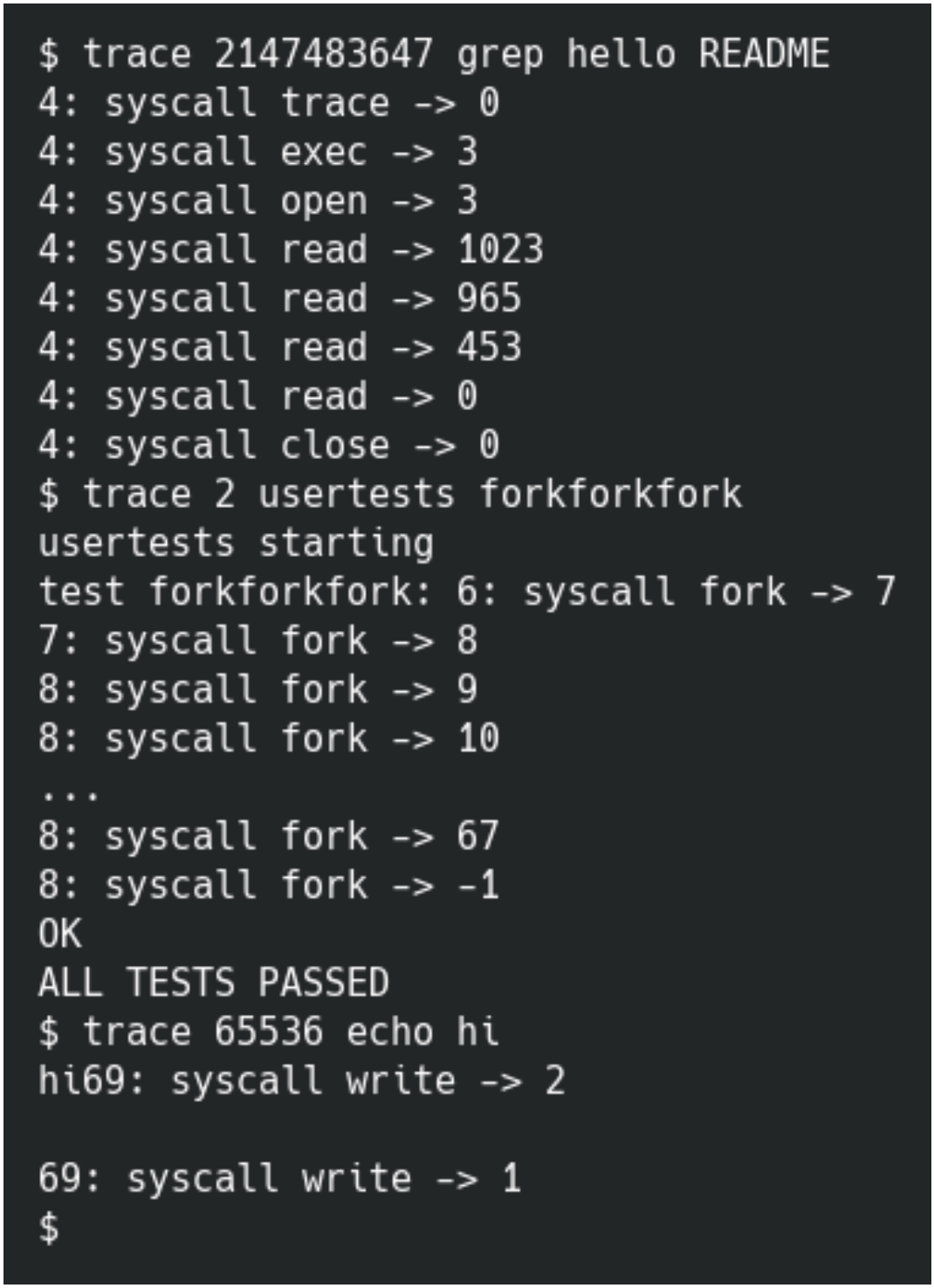


Figure 3.1 – Part 3 runs: `trace 2147483647 grep hello README`, `trace 2 usertests forkforkfork`, **and** `trace 65536 echo hi`

3.4 Answers to Part 3 observations

1. **In `trace 32 grep hello README` why do you see only `read`, and not `open` or `close`?** Mask 32 in binary has only bit 5 set ($32 = 2^5$), which corresponds to `SYS_read = 5`. Although `grep` also invokes `open` (`SYS_open = 15`) and `close` (`SYS_close = 21`) on `README` (as seen with mask `2147483647` in Section 3.1), bits 15 and 21 are `0` in mask 32, so the bitwise test `(p->trace_mask >> num) & 1` is false for those calls.
2. **In `forkforkfork`, three different PIDs print lines. Which line of code from Part 2 makes that possible?** The line `np->trace_mask = p->trace_mask;` in `fork()` (`kfork()` in `kernel/proc.c`) copies the parent's trace mask into each newly created child process. Without that line, only the initial `usertests` process (PID 6) would have `trace_mask = 2`, while its children (PID 7, PID 8, and their descendants) would have `trace_mask = 0` and would not log their `fork` calls.
3. **In `trace 65536 echo hi` the trace line is printed between `"hi"` and the new line. Why? (How many times does `echo` call `write`?)** `echo hi` calls `write` **twice**: first `write(1, "hi", 2)` to write the two characters `"hi"` (without a newline), and then `write(1, "\n", 1)` to write the newline character. Because `syscall()` prints the trace line right after each `write` system call completes, the console first displays `hi`, then immediately prints `69: syscall write -> 2` on the same line (including a newline), then the second `write` prints `\n` (producing the blank line), followed by `69: syscall write -> 1`.
4. **Which mask traces both `open` (15) and `close` (21), and nothing else?** Enabling bit 15 ($2^{15} = 32768$) and bit 21 ($2^{21} = 2097152$) gives:

$$\text{mask} = (1 \ll 15) \mid (1 \ll 21) = 32\,768 + 2\,097\,152 = \mathbf{2\,129\,920}.$$

CONCLUSIONS

Adding `SYS_trace` (number 23) to xv6 on RISC-V demonstrated the complete path of a system call across the user-kernel boundary:

- On the user side, `user/trace.c` invokes the `trace(int)` stub generated by `usys.pl`, which loads `SYS_trace` into `a7` and executes `ecall`, before calling `exec` to run the target command in the same process.
- On the kernel side, `sys_trace` records the bit mask in `myproc()->trace_mask`, `syscall()` checks `(p->trace_mask >> num) & 1` after each call to print the PID, syscall name, and return value via `printk`, and `fork()` copies `trace_mask` to child processes.
- Mask experiments in Part 3 confirmed full-process tracing (`2147483647`), multi-generation child tracing under `trace 2 usertests forkforkfork`, and per-write interleaving in `trace 65536 echo hi`.